Counter Congruence Suffices for iPRA

A structural reduction of equality rules,
with obstructions to two seedless recursion bases

Research prepared for Vladimir Reshetnikov

November 10, 2026

Abstract

With the existing successor-seeded, counter-aware recursor, congruence in its counter alone, together with replacement at the left root of an equation, derives congruence through every fixed numerical expression. The construction uses numerical cases, not equation induction, and keeps both numerical computation schemas unchanged. An inherited lifting then recovers arbitrary formula transport. The resulting complete native calculus is contextually equivalent to the existing iPRA presentation; its whole-domain, actual-equality bi-interpretation and equiconsistency are given explicitly.

The requested significant checker-complexity breakthrough is *not* established. The inspected corpus's coordinate minima are 281 normalized Python tokens, 315 AST nodes, and 685 normalized bytes, attained by different complete programs. Incidental, behavior-preserving Python changes improve these coordinates separately to 280, 314, and 672. They implement the existing theory and do not give one program below all three old minima. The new restricted calculus has no size record. Its positive contribution is fixed-counter sufficiency under the unmodified recursor equations. The numerical construction specializes an existing compiler, and the older initializer strategy is acknowledged.

Two additional theorems show why simply deleting the independent initializer is inadequate for direct term representations: zero-seeded counter recursion preserves monotonicity, while a seemingly stronger successor-seeded variant admits a unary bound depending on only one input coordinate. A third audit identifies a witness-order obstruction to a naive reversal of the paired binder's quantifiers.

Status. No significant new axiomatization below the complete checker frontier, or absolute simplestness, is established. The report supplies a complete equivalent calculus, a restriction theorem, small implementation records for the inherited theory, executable certificates, and scoped obstructions. No Lean/Rocq formalization or external human peer review is claimed.

Contents

1 The benchmark and the precise contribution **3**
1.1 What changes mathematically . . . 3
1.2 Attribution and novelty limits . . . 3

2 Complete primitive syntax and rules **4**
2.1 Numerical terms and computation . . . 4
2.2 Object formulas and internal clauses . . . 4
2.3 Restricted equality rules . . . 5
2.4 All remaining logical rules . . . 5

3 Counter congruence recovers unrestricted equality transport **5**
3.1 Structural substitution is not equality transport . . . 6
3.2 Equality paths using root replacement alone . . . 6
3.3 Numerical cases before congruence . . . 6
3.4 Representing a numerical context through the counter . . . 7
3.5 A constant bound on inference-node overhead . . . 8
3.6 Compound formulas and internal clauses . . . 9
3.7 Counter congruence alone does not recover root replacement . . . 10

4 How the equivalence claim is completed **12**

5 The short numerical construction **12**

6 Deriving the full intuitionistic logical interface **13**
6.1 Cases, equation induction, and congruence of the binder . . . 13
6.2 Numerical gates and formula selectors . . . 13
6.3 Connectives and quantifiers . . . 14

7 Full arithmetic and both interpretation composites **14**
7.1 An explicit ordinary realization . . . 14
7.2 Actual equality reflection . . . 15
7.3 The two numerical interpretations . . . 16
7.4 The logical round trips . . . 16

8 Why two seedless recursion shortcuts do not give a term compiler **17**
8.1 Zero-seeded counter recursion preserves monotonicity . . . 17
8.2 A more promising seedless operator still has only one unbounded input . . . 18

9 Witness order in the attempted reversed binder **19**

10 Implementation, certificates, and limits **20**
10.1 What the compact counter-congruence attempts show . . . 21
10.2 Incidental implementation records for the existing theory . . . 22
10.3 Reproduction . . . 23

11 Further research questions **23**

1 The benchmark and the precise contribution

The inspected ProveIt snapshot is commit

`2fdcf5bf988bead821c416649f713f98b7ed8776`.

There are nine direct ZIP files in `docs/drafts`. Each was retrieved and checked against both its byte length and Git blob identity. A recursive inventory, including the historical archives embedded inside the current packages, contains 14 distinct ZIP contents, 342 Python source occurrences, and 164 distinct Python source hashes. The inventory and the manual distinction between complete checkers and auxiliary scripts are included in `evidence/`.

Entire complete program	Tokens	AST nodes	Bytes
Generated assumptions, principal	281	318	685
Generated assumptions, AST profile	283	315	710
Guarded elimination, token profile	281	325	739
Guarded elimination, successor byte profile	290	333	689
Atomic transport, unrestricted control	297	335	698
Inspected-corpus coordinate envelope	**281**	**315**	**685**

The measurements use the unchanged inherited normalizer under CPython 3.12.14: remove comments and docstrings, apply `ast.unparse`, and count significant tokens, AST nodes, and UTF-8 bytes of the whole module. Input handling, scope checks, every inference branch, and the public verdict driver count. The host Python runtime and standard JSON library are common infrastructure. Reference-directory checkers count when they are complete programs. Tiny build and test wrappers do not.

The older 287/323/738 envelope printed in several drafts is obsolete for the snapshot examined here. Beating that older envelope would not meet the user's current target. Section 10.2 records small implementation-only improvements found during the final audit; they change the coordinate envelope to 280/314/672. These coordinates still belong to different programs. Moreover all these attained sizes are *upper bounds* on achievable checker size. They are not a theorem giving an absolute lower bound, which would require a fixed description language and cost convention.

1.1 What changes mathematically

General equality transport is normally presented with an arbitrary formula or numerical context as rule data. Here it is recovered from two much more specific operations: replacing the complete left endpoint of an equation, and changing only a recursor's counter while keeping its initializer and update fixed. The replacement of a rule by two more specific rules is a restriction of its primitive interface; it does not by itself imply shorter checker source.

The useful observation is that every fixed numerical context can be represented, provably, by two nested recursors in whose varying arguments the hole occurs only as a counter. This makes the restricted congruence rule sufficient. The proof is given before any appeal to the full iPRA interpretation, so it does not assume the equality principles being reconstructed.

1.2 Attribution and novelty limits

The successor-seeded recursor, paired binder, case/progress distinction, logical macros, ordinary arithmetic bridge, and lifting of atomic to compound transport are inherited[2, 3, 7]. The context

construction below is a specialization of the existing primitive-recursion compiler. We do not claim a new recursor, a new compiler for all primitive recursive functions, or a new compound transport theorem.

The older fixed-position report already uses root replacement and replacement in the *initializer* of a scalar iterator. Its direct context representation uses an additional first-step computation schema I1. Here the counter-aware R basis is unchanged: R0 and RS suffice, and the only congruence position needed is the *counter*. No matching restricted-rule theorem was found in the current and nested textual sources or in the separately retrieved full historical report. This is a novelty statement relative to the inspected corpus, not a claim of priority over the literature.

2 Complete primitive syntax and rules

2.1 Numerical terms and computation

Terms consist of variables, zero, and the counter-aware recursion form

$$R(n, a; u, v.h).$$

Only the update body h binds the two distinct variables u, v; its remaining parameters stay fixed through the recursion. Write

$$S(a) := R(0, a; u, v.0), \qquad 1 := S0.$$

For readability, the implementation represents S as an explicit constructor. Its defining equality with the displayed R term is already an R0 instance; it adds no independent arithmetic axiom.

The only computation schemas, available under any retained context, are

$$R(0, a; u, v.h) = S(a), \tag{R0}$$
$$R(Sn, a; u, v.h) = h[n/u, R(n, a; u, v.h)/v]. \tag{RS}$$

Substitutions are capture-free. The recursive state retains the original update h. Each body is a fixed finite term description. Equivalently, introduce a first-order symbol r_h at each finite syntactic stage. There is no object-level variable ranging over functions or programs, and no assertion of a uniformly total primitive-recursive interpreter.

2.2 Object formulas and internal clauses

Object formulas are

$$A ::= (s = t) \mid \mathsf{D}_{x,y}A,$$

where x, y are distinct bound numerical variables. A context is a finite reusable list of object formulas and internal clauses $\mathsf{Q}_y B(y)$, with B an object formula. The ordinary readings are

$$\mathsf{D}_{x,y}B \;\mapsto\; \exists x\big(B(x,0) \to \forall y\, B(x,y)\big), \qquad \mathsf{Q}_y B \;\mapsto\; B(0) \to \forall y\, B(y).$$

An internal clause is available only by context membership; it is not an exported object goal. An object formula is available either by membership or by a proof. Write this availability as $\Gamma \triangleright M$. The primitive hypothesis rule is

$$\frac{}{\Gamma \vdash A} \quad (A \in \Gamma). \tag{Hyp}$$

Here and below A as a proof conclusion is an object formula. The designated contradiction is $\bot_* = (0 = 1)$.

2.3 Restricted equality rules

$$\frac{\Gamma \triangleright s = t \qquad \Gamma \vdash s = r}{\Gamma \vdash t = r}\ \mathsf{RL}, \tag{Root replacement}$$

$$\frac{\Gamma \triangleright s = t}{\Gamma \vdash R(s, a; u, v.h) = R(t, a; u, v.h)}\ \mathsf{CC}. \tag{Counter congruence}$$

CC has exactly the same initializer, update, and passive parameters on both sides, modulo renaming of bound variables. It gives no primitive permission to alter an initializer, a successor argument, an update, or a compound formula. Reflexivity, symmetry, transitivity, and every other congruence are derived.

2.4 All remaining logical rules

$$\frac{\Gamma \triangleright \mathsf{Q}_y B(y) \qquad \Gamma \vdash B(0)}{\Gamma \vdash B(t)}\ \mathsf{QE}, \tag{Closure use}$$

$$\frac{\Gamma \triangleright \mathsf{D}_{x,y} B(x, y) \qquad \Gamma, \mathsf{Q}_y B(a, y) \vdash F}{\Gamma \vdash F}\ \mathsf{DE}, \tag{Paired elimination}$$

$$\frac{\Gamma, B(t, 0) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y} B(x, y)}\ \mathsf{DI}_{\mathrm{case}}, \tag{Case introduction}$$

$$\frac{\Gamma, E(t, z) \vdash E(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y} E(x, y)}\ \mathsf{DI}_{\mathrm{progress}}. \tag{Equation progress}$$

In DE, a is fresh for Γ, F and for all fixed parameters of the matrix. In both introduction rules, z is fresh for Γ, t and the fixed matrix parameters. Progress is permitted only when E is a *literal numerical equation*. A formula with an equation under a paired binder is compound and cannot use progress. Local hypotheses and clauses are discharged. Bound variables are renamed before all substitutions.

These rules, Hyp, R0, and RS are the entire primitive inventory. There is no primitive general equality transport, coverage axiom, separation axiom, general formula induction, excluded middle, or unlisted ambient first-order proof checker. Weakening, exchange, contraction, admissible renaming, numerical substitution, and object cut follow by external induction on finite derivations. The usual freshening of local parameters makes substitution and cut preserve all side conditions. Denote the calculus by $\mathcal{K}_{\mathrm{ctr}}$.

3 Counter congruence recovers unrestricted equality transport

The reduction in this section concerns the mathematical inference interface. A smaller primitive interface and a smaller executable checker are separate claims; only the accompanying measurements can establish the latter. We use the numerical terms, the Eq/D object formulas, the internal Q clauses, and the primitive rules specified above. In particular,

$$\begin{aligned} R(0, a; u, v.h) &= Sa, \\ R(Sn, a; u, v.h) &= h[n/u, R(n, a; u, v.h)/v], \end{aligned} \tag{R0, RS}$$

where $Sa := R(0, a; u, v.0)$. Only the update h binds u, v, and all substitutions are capture-free. No variable ranges over numerical program descriptions. A template in what follows is one fixed finite expression.

Recall the two restricted equality rules:

$$\frac{\Gamma \triangleright s = t \qquad \Gamma \vdash s = r}{\Gamma \vdash t = r}\ \mathsf{RL}, \qquad \frac{\Gamma \triangleright s = t}{\Gamma \vdash R(s,a;u,v.h) = R(t,a;u,v.h)}\ \mathsf{CC}. \qquad \text{(Root and counter)}$$

In CC the seed, update, and passive parameters are the same on both sides. The availability relation $\Gamma \triangleright M$ means that M is a retained assumption or an already proved object formula. Internal Q clauses are available by membership. Full hypothesis use is present for Eq/D object assumptions; there is no rule with an internal Q clause as its object goal.

The proof uses Hyp, QE, DE and DI-case, in addition to R0, RS, RL and CC. It does not use DI-progress. In particular, no equation-induction instance is hidden in the reduction.

3.1 Structural substitution is not equality transport

Weakening, exchange, contraction, renaming, and capture-free substitution of a numerical term for a free parameter are admissible by induction on finite derivations. Rename eigenparameters first. R0 and RS are stable under substitution; substitution preserves the matching seed and update in CC and the matching root in RL. It also preserves a formula's root constructor, so a permitted progress instance remains literal if progress occurs elsewhere in the derivation. The ordinary eigenparameter checks handle the logical rules. Object-assumption cut is obtained by replacing each use of the assumption by the supplied derivation, again freshening eigenparameters. These are operations on fixed finite proof syntax. They do not appeal to an object-theory rule replacing equal terms inside a formula.

3.2 Equality paths using root replacement alone

Lemma 3.1 (Equality algebra)**.** *Reflexivity, symmetry, and transitivity are derivable using RS and* RL*, without* CC *or general equality transport.*

Proof. For a numerical term s put

$$c_s := R(Ss, 0; u, v.u).$$

RS gives $c_s = s$. Use one copy as the available major and another as RL's minor; the conclusion is $s = s$. For symmetry, an available $s = t$ and the just-derived $s = s$ give $t = s$ by RL. For transitivity, suppose $s = t$ and $t = r$ are available or proved. Hypothesis use supplies an object proof when necessary. Symmetry gives $t = s$; apply RL with this major and minor $t = r$ to obtain $s = r$. Consequently every finite path of proved numerical equations can be composed, and any edge may be reversed. This path operation changes no proper numerical subterm. □

3.3 Numerical cases before congruence

Lemma 3.2 (Numerical cases)**.** *For any fixed Eq/*D *object template $A(y)$, the following rule is admissible without either equality rule or progress introduction:*

$$\frac{\Gamma \vdash A(0) \qquad \Gamma \vdash A(Sz)}{\Gamma \vdash A(t)}. \qquad \text{(Cases)}$$

Here z is fresh for the retained context, the fixed parameters of A, and the target term t; it may always be renamed to meet these conditions.

Proof. Choose a bound coordinate x absent from A. By weakening, the successor premise remains available under the extra assumption $A(0)$. Apply DI-case with witness zero to obtain $\Gamma \vdash \mathsf{D}_{x,y}A(y)$. Eliminate this paired formula. Because x is unused, its new internal clause is exactly $\mathsf{Q}_y A(y)$; the fresh existential parameter does not occur in that clause. Weaken the given base proof into the extended context and apply QE at t. This proves $A(t)$ there, and DE discharges the clause. There was no rewrite by an equality and no induction hypothesis. □

3.4 Representing a numerical context through the counter

Define

$$W(n) := R(n, 0; u, v.u).$$

The computations $W(0) = S0$ and $W(Sn) = n$ are instances of R0 and RS. In particular the latter identity is available before any general congruence principle has been established.

Lemma 3.3 (Counter representation of a fixed context)**.** *For any fixed numerical template $F(x)$, choose all new bound variables fresh for its passive parameters and put*

$$\begin{aligned} H_F(n) &:= R(n, F(0); u, v.S(F(Su))), \\ J_F(n) &:= W(H_F(n)). \end{aligned} \qquad \text{(Counter representation)}$$

Then the restricted calculus proves

$$J_F(n) = F(n) \qquad \text{(Representation)}$$

for arbitrary numerical n and arbitrary retained context. The proof uses numerical cases and no progress introduction.

Proof. The update of H_F does not contain its state variable v. Consequently R0 and RS give the following identities literally, with no equality substitution into an enclosing expression:

$$H_F(0) = S(F(0)), \qquad H_F(Sz) = S(F(Sz)). \qquad \text{(Two computations)}$$

For either of these instances, write its argument as q, where $q = 0$ or $q = Sz$. Apply CC to the displayed equality using seed zero and the update that returns its old counter. This produces

$$W(H_F(q)) = W(S(F(q))).$$

The right-hand expression has the computation equation $W(S(F(q))) = F(q)$. Compose the two equation paths by Lemma 3.1; the result is $J_F(q) = F(q)$. We have now supplied both premises of Lemma 3.2 for the literal equation $J_F(y) = F(y)$. Cases proves (Representation) at an arbitrary n.

Every proper-subterm change in this argument is the single, explicitly stated CC instance through W. The remaining equation compositions use root replacement. In particular one must not silently replace $H_F(q)$ by $S(F(q))$ under W before applying CC; that is precisely the congruence being justified. □

The displayed J_F is a specialization of the successor-recursion compiler already present in the inspected drafts: take its initializer to be $F(0)$ and its step to be $h(u, z) = F(Su)$, which ignores the state. The additional observation here is that this specialization is proved correct using only root replacement and congruence in the counter. The general recursion compiler itself is inherited.

Theorem 3.4 (All numerical congruence from counter congruence)**.** *For every fixed numerical template $F(x)$, the restricted calculus admits*

$$\frac{\Gamma \triangleright s = t}{\Gamma \vdash F(s) = F(t)}. \qquad \text{(Numerical congruence)}$$

Proof. Use the major $s = t$ in CC with seed $F(0)$ and update $S(F(Su))$. Its conclusion is $H_F(s) = H_F(t)$. A second CC instance, now for W, gives $J_F(s) = J_F(t)$. Lemma 3.3 supplies $J_F(s) = F(s)$ and $J_F(t) = F(t)$. Symmetry and path composition from Lemma 3.1 give

$$F(s) = J_F(s) = J_F(t) = F(t).$$

These are equations joined at their roots. Neither the proof nor the statement assumes an interpretation limited to standard numerals. □

Corollary 3.5 (Arbitrary atomic transport)**.** *For all fixed numerical templates $U(x), V(x)$,*

$$\frac{\Gamma \triangleright s = t \qquad \Gamma \vdash U(s) = V(s)}{\Gamma \vdash U(t) = V(t)} \qquad \text{(Atomic transport)}$$

is admissible.

Proof. Theorem 3.4 gives $U(s) = U(t)$ and $V(s) = V(t)$. Reverse the first equality and compose only at equation roots:

$$U(t) = U(s) = V(s) = V(t).$$

□

3.5 A constant bound on inference-node overhead

Count one node for each primitive inference, including a hypothesis lookup, and count a proof tree with repeated subproofs repeated. In the explicit expansion supplied with the report, reflexivity has three nodes. Symmetry adds four nodes to its supplied proof, and transitivity adds five nodes to the sum of its two supplied proofs.

Each of the zero and successor proofs of $J_F(q) = F(q)$ has eight nodes: a computation, one CC lifting, a second computation, and the five-node transitivity overhead. The cases construction consists of DE, DI-case, QE, one clause hypothesis lookup, and these two eight-node proofs. Thus $J_F(t) = F(t)$ has twenty nodes.

If the supplied proof of $s = t$ has P nodes, the two CC liftings have $P + 2$ nodes, the reversed left representation has $20+4$ nodes, and the right representation has twenty. Two transitivity constructions therefore give

$$(P + 2) + (20 + 4) + 20 + 5 + 5 = P + 56.$$

The additional inference-node count is independent of the syntactic depth of the fixed numerical template F. Freshening binders changes names, not the number of inferences. This is not a constant byte-size or running-time bound: the proof's term and formula labels contain copies of F and of the supplied endpoints. It is also not a constant bound for the subsequent compound-formula lifting. The delivered successor, initializer, and update-body congruence certificates each have 57 nodes when their supplied equality is a one-node computation.

3.6 Compound formulas and internal clauses

The following lifting pattern is inherited from the earlier fixed-position and atomic-transport arguments. It is included to show that the numerical result recovers the full logical interface, and to make its independence from compound induction explicit.

Theorem 3.6 (Full object-formula transport)**.** *For every fixed Eq/*D *object template* $A(w)$*, the rule*

$$\frac{\Gamma \triangleright s = t \qquad \Gamma \vdash A(s)}{\Gamma \vdash A(t)} \qquad \text{(Object transport)}$$

is admissible. Its expansion introduces no progress instances.

Proof. Induct externally on the number of paired binders in A. The induction statement is uniform in all retained contexts, source terms, and passive parameters. For a literal equation, use Corollary 3.5.

Suppose $A(w) = \mathsf{D}_{x,y}B(w, x, y)$. Rename the two bound variables away from s, t and the retained context. Eliminate the given $A(s)$ at a fresh parameter a. The extended context retains

$$\mathsf{Q}_y B(s, a, y).$$

To obtain the target $\mathsf{D}_{x,y}B(t, x, y)$, use DI-case with witness a and a fresh successor parameter z. Its local object assumption is $B(t, a, 0)$; hypothesis use gives an object proof of it. Symmetry gives $t = s$. The external induction hypothesis applied backwards to the smaller template $B(w, a, 0)$ therefore gives $B(s, a, 0)$. Specialize the retained clause at Sz to obtain $B(s, a, Sz)$. Apply the induction hypothesis forwards along $s = t$, now to $B(w, a, Sz)$, obtaining $B(t, a, Sz)$. This is exactly the required case-introduction premise. Close DI-case and then DE.

The new parameter z is absent from the retained clause, a, and all fixed matrix parameters; the eliminated witness a is absent from the outer conclusion. Both recursive transport uses concern the strictly smaller object matrix B. This is recursion on finite external syntax, not an induction rule of the arithmetic. The only introductions added in the binder case are case introductions. □

Proposition 3.7 (Transporting the use of an internal clause)**.** *An available clause* $\mathsf{Q}_y B(s, y)$*, together with an available* $s = t$*, can be used with exactly the operational force of* $\mathsf{Q}_y B(t, y)$*: from any object proof of* $B(t, 0)$ *one can obtain* $B(t, r)$ *for any numerical* r*.*

Proof. Transport the supplied base backwards along $t = s$, specialize the old clause at r, and transport the result forwards along $s = t$, using Theorem 3.6 for the two object matrices. This statement does not add a rule proving an internal clause as an object formula. If an object representative is required, use $\mathsf{D}_{x,y}B(t, y)$ with unused x; case introduction and the just-described clause use construct that representative. □

Corollary 3.8 (Exact identity-language equivalence)**.** *Let* $\mathcal{T}_{\mathrm{ctr}}$ *have R0, RS, object hypothesis use,* RL*,* CC*,* QE*,* DE*, and the case and literal-equation progress introductions. Let* $\mathcal{T}_{\mathrm{tr}}$ *instead use unrestricted Eq/*D *object transport as its equality rule, with all other numerical and logical syntax and rules the same. For every finite context of object assumptions and internal clauses, and every object formula* A*,*

$$\Gamma \vdash_{\mathcal{T}_{\mathrm{ctr}}} A \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{T}_{\mathrm{tr}}} A.$$

The translations are effective on finite derivations and preserve their end sequents literally.

Proof. From right to left, translate premise derivations recursively and replace each unrestricted transport instance by Theorem 3.6. All other rules are copied. From left to right, RL is the unrestricted-transport instance with motive $w = r$. Reflexivity is derived from a computation equation by that same root instance. For CC, begin with reflexivity $R(s, a; h) = R(s, a; h)$ and transport its left occurrence of s to t; this gives $R(t, a; h) = R(s, a; h)$. Derived symmetry gives the required orientation. The other rules are again copied. All side conditions are preserved by freshening local parameters. □

Because this equivalence leaves the native language and all primitive meanings unchanged, the whole-domain bi-interpretations and actual-equality composite isomorphisms of $\mathcal{T}_{\mathrm{tr}}$ transfer to $\mathcal{T}_{\mathrm{ctr}}$. A proof of that arithmetic and logical bridge is supplied separately; exact sequent equivalence is not a substitute for stating the bridge. The present argument supplies no absolute lower bound on checker size and no claim of novelty outside the specifically inspected sources.

3.7 Counter congruence alone does not recover root replacement

The two restricted equality rules are not interchangeable. The following counterinterpretation shows that simply deleting root replacement loses even the theorem $0 = 0$. This is a statement about the explicitly specified native inference system. It does not rule out every different rule, encoding, numerical basis, or interpretation.

Theorem 3.9 (Root replacement is not derivable from the remaining rules)**.** *Let* $\mathcal{T}^{-\mathsf{RL}}$ *have the same terms, Eq/*D *object formulas, internal* Q *clauses, and rules R0, RS,* Hyp*,* CC*,* QE*,* DE*,* DI*-case, and* DI*-progress as the counter-congruence calculus, but omit* RL*. Do not add an unlisted reflexivity, equality-substitution, or other ordinary equality rule. Then*

$$\varnothing \nvdash_{\mathcal{T}^{-\mathsf{RL}}} 0 = 0.$$

In particular, RL *is not admissible in this reduced system.*

Proof. For this syntactic nonderivability argument, interpret the native atom written $s = t$ by a binary predicate E. We are not assuming that this predicate is actual equality: the missing equality principles are precisely what the proposed reduction would have to derive.

Take numerical domain $\{0, 1\}$. Interpret zero as 0 and every recursion expression by

$$[\![R(n, a; u, v.h)]\!]_\rho = 1$$

under every assignment ρ, independently of its counter, seed, update, and passive parameters. This is compatible with capture-free substitution and alpha-renaming. The successor abbreviation $Sa = R(0, a; u, v.0)$ is therefore the constant function 1, and the numeral $1 = S0$ denotes the domain element 1. Interpret the equality-shaped atom by

$$[\![s = t]\!]_\rho \text{ is true} \quad \Longleftrightarrow \quad [\![s]\!]_\rho = 1. \qquad \text{(Predicate interpretation)}$$

Its right-hand term is immaterial. The equality on the right of this metatheoretic definition is the usual equality of the two domain elements. Interpret the binders and internal clauses by their ordinary meanings:

$$(\mathsf{Q}_y B)^\circ := B^\circ(0) \to \forall y\, B^\circ(y),$$
$$(\mathsf{D}_{x,y} B)^\circ := \exists x (B^\circ(x, 0) \to \forall y\, B^\circ(x, y)).$$

These are ordinary two-valued semantics, equivalently a one-world intuitionistic interpretation.

We check every retained primitive rule under arbitrary assignments and arbitrary true retained contexts.

Computation and counter congruence. Each R0 or RS axiom has an R expression as the left side of its atom. That expression has value 1, so the atom is true, irrespective of its right side. A CC conclusion also has an R expression on its left and is therefore true. These rules are sound even though the binary predicate is not reflexive at zero.

Hypotheses, availability, and elimination. An object hypothesis is true whenever its retained context is true. An available major is either such a retained assumption or an already proved object formula, so availability preserves truth. For QE, a true clause $B(0) \to \forall y B(y)$ and a true base $B(0)$ give its requested instance $B(t)$. For DE, the true existential major supplies a domain element satisfying the retained closure clause. The fresh witness parameter can be assigned that element; its absence from the outer conclusion and retained context gives the ordinary soundness argument for existential elimination.

Case introduction for arbitrary object matrices. Fix an assignment making Γ true, and let b be the value of the submitted witness t. Freshness makes b and Γ independent of the new successor parameter z. A sound premise

$$\Gamma, B(t,0) \vdash B(t,Sz)$$

ensures that $B(b,0)$ implies $B(b,1)$, since Sz always denotes 1. The domain has just 0 and 1, so this implication ensures $B(b,0) \to \forall y B(b,y)$. Choosing $x = b$ proves the intended D conclusion. This verifies DI-case without restricting its matrix to an equation.

Progress introduction for literal equations. Fix the passive parameters and the witness. By inspection of finite numerical term syntax, a term $U(y)$ has one of the following three value functions of the remaining coordinate y:

$$y \longmapsto 0, \qquad y \longmapsto 1, \qquad y \longmapsto y.$$

Indeed a zero term is constant 0; a fixed passive variable is constant 0 or 1; the distinguished variable has value y; and any term with an R root, including a successor abbreviation, is constant 1. For a literal equation $E(y) = (U(y) = V(y))$, its truth depends only on $U(y)$ by the predicate interpretation. Its truth set is consequently one of

$$\varnothing, \qquad \{1\}, \qquad \{0,1\}.$$

In every case, truth of $E(0)$ implies truth of $E(y)$ for both domain elements. Thus the required closure $E(0) \to \forall y E(y)$ is valid for every fixed witness, even before using the progress premise. This proves soundness of DI-progress. The restriction to a literal equation is essential to this verification; it is not a claim about arbitrary compound matrices.

All primitive rules of $\mathcal{T}^{-\mathsf{RL}}$ are now sound. Induction on a finite derivation therefore preserves truth in this interpretation. But the atom $0 = 0$ is false because its left term has value 0. It has no derivation from the empty context.

Finally, in the full calculus RS gives

$$R(S0, 0; u, v.u) = 0.$$

Two copies of this computation, used as the major and minor of RL, prove $0 = 0$. If RL were admissible in the reduced system, this three-node argument could be translated into it, contradicting the counterinterpretation. □

The conclusion is deliberately limited. Deleting RL destroys the established identity-language equivalence with the unrestricted transport calculus. The counterinterpretation is not asserted to be a model of an ordinary theory in which equality already carries its usual logical axioms, and it is not an impossibility theorem for all bi-interpretations with a different equality or domain. Importing those usual equality axioms silently would change the proof-checker problem.

4 How the equivalence claim is completed

The new restriction theorem proves exact consequence equivalence with the inherited calculus, using the identity map on numerical terms, object formulas, and contexts. This gives more than a standard-model simulation: every finite proof can be translated in both directions. To make the stronger bi-interpretability claim reviewable, the next three sections give the arithmetic and logical bridge in full.

Those sections adapt the atomic-transport report at the pinned snapshot, with attribution[2]. Every invocation of general transport is justified by the preceding theorem; every object assumption is available by Hyp. The earlier report's elimination-only implementation is not silently imported. In particular, the two interpretation composites are checked for arbitrary elements and fixed descriptions, with equality given by actual numerical equality.

5 The short numerical construction

The following direct compiler is inherited from the successor-recursion draft[4]; it is included to make the present construction self-contained. Atomic equality reasoning already gives transitivity and numerical congruence. Define

$$W(n) := R(n, 0; u, v.u), \qquad C(n, a, b) := W(R(n, a; u, v.Sb)), \qquad P(n) := C(n, 0, W(n)).$$

Computation and derived equality reasoning prove

$$\begin{aligned} W0 &= 1, & W(Sn) &= n, \\ C(0, a, b) &= a, & C(Sn, a, b) &= b, \\ P0 &= 0, & P(Sn) &= n. \end{aligned}$$

The counter input u of R is essential: it is not an alias for v. Applying W proves that successor is injective.

Suppose fixed terms $g(\vec{p})$ and $h(n, z, \vec{p})$ are already represented. Put

$$J_{g,h}(n, \vec{p}) = W\big(R(n, g(\vec{p}); u, v.Sh(u, W(v), \vec{p}))\big). \qquad \text{(Compiler)}$$

Then

$$J_{g,h}(0, \vec{p}) = g(\vec{p}), \qquad J_{g,h}(Sn, \vec{p}) = h(n, J_{g,h}(n, \vec{p}), \vec{p}).$$

At zero, R returns Sg and W removes the successor. At a successor, R returns $Sh(n, J_{g,h}(n, \vec{p}), \vec{p})$ and W again removes the displayed successor. Both identities hold at arbitrary source elements; there is no standard-numeral restriction or positivity premise. Zero, successor, and projections are terms; composition is substitution. External recursion on any fixed PR description therefore compiles it. No scalar predecessor search, pairing construction, or equation induction is needed for this compiler. Scalar iteration is its special case with h independent of its counter.

6 Deriving the full intuitionistic logical interface

This section adapts the inherited Choice-DI interface[5, 4]. Every retained object formula is available by the Hyp rule, and every compound equality transport means Theorem 3.6. Thus none of the following reasoning adds a primitive checker rule.

6.1 Cases, equation induction, and congruence of the binder

With an unused coordinate x, progress DI introduces $\mathsf{D}_{x,y}E(y)$ from $\Gamma, E(z) \vdash E(Sz)$ for a literal equation E. DE opens its clause $\mathsf{Q}_y E(y)$; a supplied base proof $E(0)$ and QE at t give $E(t)$. Hence contextual equation induction is admissible:

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)}. \tag{EI}$$

Case DI in the same construction, ignoring its base assumption in a given successor proof, yields numerical cases for any object template:

$$\frac{\Gamma \vdash A(0) \qquad \Gamma \vdash A(Sz)}{\Gamma \vdash A(t)}. \tag{NC}$$

The usual freshness conditions are retained. Witness 1 may be used for the unused coordinate. These derivations never need a proof whose conclusion is an internal Q clause.

Uniform pointwise interderivability of matrices is preserved under D. Eliminate the source D at a fresh witness, introduce the target D at that witness by case DI, convert its base backwards, use the opened source clause at Sz, and convert the result forwards. This is the same constructive pattern as the transport theorem, now with arbitrary uniform conversion derivations. Reverse the conversions for the converse.

6.2 Numerical gates and formula selectors

For a numerical control z define, externally on fixed formula syntax,

$$G_z(s = t) := (C(z, 0, s) = C(z, 0, t)), \qquad G_z(\mathsf{D}_{x,y}B) := \mathsf{D}_{x,y}G_z B.$$

Freshen binders away from the control. Computation, reflexivity, and binder congruence prove $\vdash G_0 A$ and $G_{Sz}A \dashv\vdash A$. The padding operation $\mathrm{Pad}(A) = \mathsf{D}_{u,y}G_y A$, with unused u,y, is equivalent to A: case DI proves the forward direction; DE, the true zero gate, and QE at 1 prove the reverse direction.

Define the selector on two equation leaves by

$$M_z(s = t, r = w) := (C(z, s, r) = C(z, t, w)).$$

At aligned fresh binders extend it by

$$\begin{aligned} M_z(\mathsf{D}_{x,y}A, \mathsf{D}_{x,y}B) &= \mathsf{D}_{x,y}M_z(A, B), \\ M_z(E, \mathsf{D}_{x,y}B) &= \mathsf{D}_{x,y}M_z(G_y E, B), \\ M_z(\mathsf{D}_{x,y}A, E) &= \mathsf{D}_{x,y}M_z(A, G_y E), \end{aligned}$$

where E is an equation. The maximum input binder depth decreases in each recursive call. Padding and binder congruence give

$$M_0(A, B) \dashv\vdash A, \qquad M_{Sz}(A, B) \dashv\vdash B.$$

These are finite syntactic operations, not object-language function symbols.

6.3 Connectives and quantifiers

Use the scalar iteration I compiled by (Compiler), put $Z(n) = C(n, 1, 0)$ and $p(n) = I(n, 0; v.Z(v))$. The only parity facts needed are the finite computations $p(1) = 1$ and $p(2) = 0$. With fresh bound variables and unused u define

$$\begin{aligned} A \Rightarrow_* B &= \mathsf{D}_{u,z} M_z(A, B), \\ A \wedge_* B &= \mathsf{D}_{u,z} G_z M_{p(z)}(A, B), \\ \exists_* x A(x) &= \mathsf{D}_{x,y} G_y A(x), \\ A \vee_* B &= \exists_* x M_x(A, B), \\ \forall_* x A(x) &= A(0) \wedge_* \mathsf{D}_{u,z} G_z A(z), \\ \bot_* &= (0 = 1). \end{aligned}$$

Implication introduction converts its retained zero selector into A, applies the supplied derivation of B, and converts B to the successor selector; case DI closes. Elimination is DE followed by QE at 1. Existential introduction uses its supplied witness and the successor gate. Elimination opens D, supplies the theorem given by its zero gate, and uses QE at 1. These are the usual eigenparameter conditions.

Disjunction introductions use control witnesses 0 and 1. For elimination, turn the two branch derivations into $A \Rightarrow_* F$ and $B \Rightarrow_* F$. NC gives $M_z(A, B) \Rightarrow_* F$ at any control; eliminate the existential selector and apply the implication. For conjunction introduction NC gives $M_v(A, B)$ at every v from the two supplied proofs. Specialize to $p(Sz)$, add the successor gate, and use case DI. Projections are QE at 1 and 2 after DE, using the two displayed parity values.

NC now gives the coverage formula

$$(x = 0) \vee_* \exists_* y\, (x = Sy).$$

At zero use the left injection of reflexivity; at a successor use the right injection with its evident witness. Coverage permits a case split even if x occurs in retained assumptions: it adds an equality naming the case, rather than illicitly substituting into the whole context. Universal introduction obtains the zero and successor cases of a fresh-parameter proof and forms the two conjuncts. Universal elimination uses coverage, the first conjunct at zero, and the gated closure at a successor. This avoids an eigenvariable assumption about the retained context.

Finally, under $0 = 1$, computation and atomic transport prove

$$s = C(0, s, t) = C(1, s, t) = t$$

for arbitrary s,t. Every equation follows; external recursion on object syntax, using case DI at each D, proves every object formula. This derives explosion for the designated absurdity. We have thus derived the full intuitionistic natural-deduction rules, with actual equality, without postulating excluded middle for quantified formulas.

7 Full arithmetic and both interpretation composites

7.1 An explicit ordinary realization

Let T_R be ordinary intuitionistic first-order logic with equality in the staged R signature, the equations R0 and RS, contextual literal-equation induction EI, and

$$\forall x\, (x = 0 \vee \exists y\, x = Sy), \qquad (0 = 1) \to \bot. \tag{Boundary}$$

For definiteness, ordinary logic here has hypothesis, implication introduction and modus ponens, conjunction introduction and its two projections, the two disjunction introductions and case elimination, universal introduction and specialization, existential introduction and fresh-witness elimination, and explosion. Equality has reflexivity and substitution. The usual eigenvariable restrictions are imposed. These ordinary rules specify the comparison theory, not additional primitives of $\mathcal{K}_{\mathrm{ctr}}$.

Realize its object and clause syntax by

$$(s = t)^\circ = (s = t), \quad (\mathsf{D}_{x,y}B)^\circ = \exists x(B^\circ(x, 0) \to \forall y B^\circ(x, y)),$$

$$(\mathsf{Q}_y B)^\circ = B^\circ(0) \to \forall y B^\circ(y).$$

The interpretation domain is $x = x$ and equality is actual equality. Case DI is sound by coverage. Progress DI is sound by equation induction under its temporarily assumed base. All other native rules have their direct ordinary meanings. Conversely Section 6 derives the ordinary rules, coverage, and EI. Separation translates to the identity implication $(0 = 1) \Rightarrow_* (0 = 1)$.

The explicit falsity bridge matters. The positive numerical equations alone admit a singleton structure, so declaring them an ordinary first-order theory while silently omitting separation would be a mistake.

7.2 Actual equality reflection

For clarity, the remaining arithmetic bridge is given internally, not merely by agreement on standard numerals. Every fixed PR description is already compiled by Section 5. In particular use

$$x + 0 = x, \quad x + Sy = S(x + y), \qquad d(x, 0) = x, \quad d(x, Sy) = P(d(x, y)).$$

Let $\mathrm{sg}(x) = C(x, 0, 1)$. Coverage and the computation equations show that sg takes only 0,1, that $\mathrm{sg}(x) = 0 \leftrightarrow x = 0$, and $C(x, a, a) = a$. Applying sg and separation gives $Sx \neq 0$. Coverage and W give $W(x) = 0 \leftrightarrow x = 1$.

Literal-equation induction proves, in order,

$$0 + y = y, \qquad (Sx) + y = S(x + y), \qquad d(Sx, Sy) = d(x, y), \qquad d(Sa, a) = 1.$$

For the first two induct on y with x fixed. For the third induct on y: the base is $P(Sx) = x$, and its step applies P to the induction equation. For the fourth induct on a and use the third identity in the step.

One more equation induction, on y with x fixed, gives

$$C(\mathrm{sg}(d(x, y)), x, y + d(x, y)) = x. \tag{Residual}$$

The base is $C(\mathrm{sg}(x), x, x) = x$. In the step set $r = d(x, y)$ and $r' = Pr = d(x, Sy)$. Coverage splits r' into zero or successor. At zero the selector already returns x. In the successor case, $r' \neq 0$ forces r to be a successor, and $P(Sq) = q$ gives $r = Sr'$. The induction equation says $y + r = x$. The new second branch is $(Sy) + r' = y + Sr' = y + r = x$. This completes the induction without using a decidable actual-equality predicate in its premises.

Consequently $d(Sa, b) = 1 \leftrightarrow a = b$: forwards, Residual yields $b + 1 = Sa$, hence $Sb = Sa$, and W cancels the successors; backwards use $d(Sa, a) = 1$. Thus

$$e(a, b) = \mathrm{sg}(W(d(Sa, b)))$$

is Boolean and satisfies $e(a, b) = 0 \leftrightarrow a = b$ for arbitrary elements of every source model.

For a fixed quantifier-free formula A, define its zero-true characteristic by

$$b_{s=t} = e(s,t), \qquad b_{\bot} = 1,$$
$$b_{A\wedge B} = C(b_A, b_B, 1), \qquad b_{A\vee B} = C(b_A, 0, b_B), \qquad b_{A\to B} = C(b_A, b_B, 0).$$

An external induction on A, using the proved Boolean ranges, gives $b_A \in \{0,1\}$ and $b_A = 0 \leftrightarrow A$ intuitionistically. The implication case uses that $b_A = 1$ proves $\neg A$; it does not assert excluded middle for arbitrary quantified formulas. Applying EI to $b_A(n) = 0$ recovers every contextual quantifier-free induction instance.

7.3 The two numerical interpretations

Take iPRA in its full first-order language of fixed PR descriptions, with defining equations, constructor separation, and quantifier-free induction. Its atomic-induction formulation is equivalent by the characteristic argument above; this is the usual first-order iPRA convention[8]. Interpret its descriptions in T_R by (Compiler), leaving domain, equality, and ordinary logic unchanged. All defining equations, successor axioms, and induction instances have just been verified internally.

Conversely, each fixed R body is interpreted in iPRA by ordinary primitive recursion with base Sa and step $h(u, v, \vec{p})$. The abbreviation $S(a) = R(0, a; u, v.0)$ then has ordinary successor value. R0, RS, the boundary sentences, and EI are iPRA theorems.

For the stronger bi-interpretability claim, first anchor the successor abbreviation. Its iPRA interpretation is the zero instance of the recursor with zero update, so its defining base clause gives ordinary Sa. Compiling that interpreted successor back gives the original $S(a)$ by the compiler's base computation. This includes the expansion $r_0(0, a)$ used to name successor, and does not presuppose uniqueness of all R symbols. Thus the same successor occurs in both copies of each recursion before the following staged uniqueness argument begins.

Now fix independent initializer and parameter variables. Two solutions of the same fixed R recursion are internally equal by induction on their value equation: bases agree, and their steps agree by equality congruence and the induction equation. Induct externally on the finite staged description. Its double translation satisfies the same equations once earlier subdescriptions have been identified, so uniqueness identifies it with the original. The identical argument works for ordinary PR descriptions. Only after these identities have been proved for independent parameters do we substitute counter-dependent actual arguments. This also establishes coherence when substitution changes the name of a staged body.

Both numerical interpretation domains are the entire original domain; both internally definable isomorphisms have graph $x = y$. Nothing is restricted to the standard cut or quotiented by a numerical observer.

7.4 The logical round trips

The ordinary meanings of gates and selectors give the advertised meanings of all six macros in Section 6, with coverage in the universal case and separation for falsity. For the only primitive object binder the reverse composite is explicitly

$$\mathsf{D}_{x,y}B \dashv\vdash \exists_* x\big(B(x,0) \Rightarrow_* \forall_* y B(x,y)\big).$$

Forwards, open D at fresh a, use its available clause under the assumed base to prove each $B(a, y)$, form the conditional universal, and introduce the existential at a. Backwards, eliminate the existential

at fresh a, introduce the original D by case DI, and use its retained base, the conditional, and universal specialization to obtain the successor case. Neither direction requires Q to be an object goal. External induction on formulas establishes both logical composites.

Theorem 7.1 (Whole-domain bi-interpretation)**.** *The ordinary realization of $\mathcal{K}_{\mathrm{ctr}}$ is bi-interpretable with full first-order iPRA. The two numerical isomorphisms are internally given by $x = y$, and both logical formula composites are provably equivalent to their originals. Effective finite proof translations preserve the designated contradiction, hence the two theories are equiconsistent.*

This is an external metatheorem. No claim is made that either theory proves its own consistency or the uniform correctness of an arbitrary primitive-recursive evaluator.

8 Why two seedless recursion shortcuts do not give a term compiler

The two results below concern the standard numerical functions denoted by fixed finite terms. They do not rule out every possible relational interpretation, quotient interpretation, or change of domain. In particular, failure to define a function by a term is weaker than failure to define its graph and prove its totality. The results nevertheless exclude the straightforward term-translation strategy proposed for these two reduced numerical grammars.

8.1 Zero-seeded counter recursion preserves monotonicity

Consider numerical terms generated by variables, zero, successor, and

$$L(n; u, v.h),$$

where only the update h binds u, v. Give each fixed update its standard natural-number interpretation by

$$L(0; u, v.h) = 0, \qquad L(Sn; u, v.h) = h[n/u, L(n; u, v.h)/v]. \qquad \text{(Zero-seeded recursion)}$$

The update may contain passive parameters and smaller recursion terms; there is no quantified function input.

Proposition 8.1 (Monotonicity obstruction)**.** *Every standard function denoted by a term of this grammar is coordinatewise nondecreasing. Therefore its terms cannot denote all primitive-recursive functions; in particular they cannot denote the zero indicator $z(0) = 1$, $z(Sn) = 0$.*

Proof. Induct on finite term syntax. Variables, zero, and successor have the required property. For the recursion constructor, the induction hypothesis makes the standard update function $h(u, v, \vec{p})$ coordinatewise nondecreasing. With $\vec{p}$ fixed, write

$$r_0 = 0, \qquad r_{k+1} = h(k, r_k, \vec{p}).$$

We have $r_0 \leq r_1$ because all values are natural numbers. If $r_k \leq r_{k+1}$, monotonicity of h in its first two coordinates gives

$$r_{k+1} = h(k, r_k, \vec{p}) \leq h(k+1, r_{k+1}, \vec{p}) = r_{k+2}.$$

Thus the value is nondecreasing in its counter. Separately, induction on k shows $r_k(\vec{p}) \leq r_k(\vec{p}')$ whenever $\vec{p} \leq \vec{p}'$ coordinatewise. Combining the two comparisons proves joint monotonicity in the counter and passive parameters. Substituting the actual counter term and parameter terms preserves this property by the structural induction hypothesis. The zero indicator violates monotonicity at $0 < 1$. □

8.2 A more promising seedless operator still has only one unbounded input

Now consider the grammar with variables, zero, and $B(n; u, v.h)$, defined by

$$\begin{aligned} B(0; u, v.h) &= S(h[0/u, 0/v]), \\ B(Sn; u, v.h) &= h[n/u, B(n; u, v.h)/v], \qquad \text{(Successor at the base)} \\ Sa &:= B(0; u, v.a). \end{aligned}$$

The last line is an abbreviation and agrees with ordinary successor in the intended standard interpretation. The base value is tied to the same update that governs the successor steps; there is no independent initializer argument.

This operator escapes the previous monotonicity obstruction. For example, the following standard identities hold:

$$\begin{array}{lll} W(n) := B(n; u, v.u), & W(0) = 1, & W(Sn) = n, \\ Z(n) := B(n; u, v.0), & Z(0) = 1, & Z(Sn) = 0, \\ H(n, a) := B(n; u, v.a), & H(0, a) = Sa, & H(Sn, a) = a. \end{array}$$

In particular $P(n) := W(H(n, n))$ is the ordinary truncated predecessor: $P(0) = 0$ and $P(Sn) = n$. Even a data gate is available. Define

$$G(n, a) := P(P(B(a; u, v.H(n, v)))).$$

At $n = 0$ the internal recursion starts at 2 and applies successor, so its value is $a + 2$ and $G(0, a) = a$. At $n = Sq$ it starts at 1 and applies the identity, so $G(Sq, a) = 0$. These standard computations are also formalizable using equation induction in an ordinary presentation of the displayed equations. They do not, however, supply a general selector or initialized primitive recursion.

Theorem 8.2 (One-coordinate unary bound)**.** *Every fixed term has a bound depending on at most one input. Precisely, for a term $t(x_1, \ldots, x_k)$ of the displayed B grammar, one of the following holds in the standard natural numbers:*

1. *t has a constant bound independent of all inputs;*
2. *for some coordinate i and some unary primitive-recursive function F, fixed by the term,*

$$t(x_1, \ldots, x_k) \le F(x_i) \quad \text{for every } (x_1, \ldots, x_k) \in \mathbb{N}^k.$$

The coordinate and its bounding function may differ between terms.

Proof. Induct on the finite syntax. The variable and zero cases are immediate; the optional successor abbreviation also preserves the claim. At a recursion term $B(n; u, v.h)$, the induction hypothesis applies to h as a term in the two distinct bound variables u, v and its passive parameters. Let r_j denote the standard recursive values at a fixed assignment of passive parameters.

If $h(u, v, \vec{p}) \le F(p_i)$ for a passive coordinate, then

$$r_0 = 1 + h(0, 0, \vec{p}) \le 1 + F(p_i), \qquad r_{j+1} = h(j, r_j, \vec{p}) \le F(p_i).$$

Hence $1 + F(p_i)$ is a bound independent of the counter and every other parameter. A constant bound on h is handled in exactly the same way.

If the controlling coordinate of h is its counter, so $h(u, v, \vec{p}) \le F(u)$, define the nondecreasing unary function

$$M(q) := 1 + \max_{j \le q} F(j).$$

The base and every successor value at counter q are bounded by $M(q)$. By the induction hypothesis the actual counter term n is bounded by a constant or by a unary function $N(x_i)$. Monotonicity of M gives respectively the constant bound $M(c)$ or the one-coordinate bound $M(N(x_i))$.

Finally, suppose the controlling coordinate of h is its state: $h(u, v, \vec{p}) \le F(v)$. Put

$$b := 1 + F(0), \qquad K(m) := m + 1 + \max_{j \le m} F(j).$$

The function K is nondecreasing, dominates F, and satisfies $K(m) \ge m$. Ordinary numerical induction in the metatheory gives

$$r_q \le K^q(b).$$

Indeed the base is bounded by b, and the step follows from $r_{q+1} \le F(r_q) \le K(r_q) \le K(K^q(b))$. Since $K \ge \mathrm{id}$, the bound $q \mapsto K^q(b)$ is nondecreasing. Compose it with the constant or one-coordinate bound on the actual counter term n, as in the preceding case.

Bounded maxima, fixed-function iteration, and composition of fixed primitive-recursive descriptions are primitive recursive. Thus each bound used here is a unary primitive-recursive function. These bounding functions belong to the ordinary metatheory; the argument does not assert that they are terms of the proposed reduced grammar. Every possible controlling coordinate of h has been handled, completing the structural induction. □

Corollary 8.3 (Addition and selection)**.** *The displayed B grammar has no term for ordinary addition $(a, b) \mapsto a + b$ and no term for the selector*

$$C(c, a, b) = \begin{cases} a, & c = 0, \\ b, & c > 0. \end{cases}$$

Consequently it cannot provide a term interpretation of the full primitive-recursive signature by the proposed direct compiler.

Proof. Addition cannot have a bound depending on just a: hold a fixed and let b increase. The symmetric argument excludes a bound depending only on b; a constant bound is also impossible.

For the selector, a bound depending only on c fails at $c = 0$ as a increases. A bound depending only on a fails when a is fixed, $c = 1$, and b increases. A bound depending only on b fails when b is fixed, $c = 0$, and a increases. These exhaust its input coordinates, contradicting Theorem 8.2. □

The obstruction is stronger than an unsuccessful search for a selector. It applies to every finite term of this grammar, including arbitrary nesting of its recursion constructor and the displayed predecessor, zero indicator, and gate as abbreviations. The gate is consistent with the theorem because it is bounded by its one data argument a. The independent initializer in the established R basis permits the initial state and the iteration length to carry two independent unbounded inputs; the above argument explains why deleting that initializer is a substantive loss for this particular term language.

9 Witness order in the attempted reversed binder

One tempting simplification replaces the inherited $\exists x\, \forall y$ order by

$$\mathsf{E}_{x,y} B := \forall y\, \exists x (B(x, 0) \to B(x, y)).$$

The hope is to combine the two eliminations and remove the reusable internal Q clause. Dummy coordinates and numerical gates still suggest formulas for implication, existence, and universality. That observation alone is insufficient for bi-interpretability.

Consider the naive case rule that chooses a fixed numerical witness term $t(z)$ as part of the introduction instruction and then checks $B(t(z), 0) \vdash B(t(z), Sz)$. Its progress variant uses a witness independent of z and a literal equation. The source existential witness obtained at a fresh universal parameter may need to be chosen *inside* that parameter's proof scope, after an existential elimination. Choosing it before the subproof does not permit this interleaving.

There is a semantic obstruction to the intended reverse formula composite. On standard $\mathbb{N}$, interpret the raw E binder by

$$\text{there exists a unary primitive-recursive } f \text{ such that } \forall y\,[B(f(y), 0) \to B(f(y), y)]. \tag{PR-selector reading}$$

Free parameters are held fixed. The naive case rule is sound: choose $f(0) = 0$ and $f(Sz) = t(z)$. The fixed-witness progress rule is sound with constant f, and the ordinary instance elimination is sound. Equality transport is sound under actual equality. The gated macros for the ordinary logical connectives have their usual two-valued meanings: an existential macro needs only a constant selector, while the universal macro has an unused existential coordinate.

Take a total computable non-primitive-recursive function F. Such a function can be obtained by diagonalizing over finite unary PR descriptions. Let $T(n, c)$ be the primitive-recursive predicate saying that c is a finite halting computation trace of a fixed program for F on input n. Verification of a bounded finite trace and extraction of its output are primitive recursive. Choose a literal equation matrix B with

$$B(x, 0) \equiv \top, \qquad B(x, Sn) \equiv T(n, x).$$

It exists using a primitive-recursive characteristic and a numerical case selector. Its ordinary $\forall y \exists x$ reading is true. But (PR-selector reading) is false: a PR selector would supply the trace $f(Sn)$, whose PR output decoder would compute $F(n)$.

Thus the naive rules do not derive the desired equivalence between every raw E formula and its ordinary reading. The argument concerns formula equivalence under assumptions; it does not need iPRA to prove the totality of F. A valid repair must permit delayed witness choice or an intermediate introduction judgment, and count its implementation cost. This excludes this naive rule package with this intended reading, not every possible theory using an E-shaped connective.

10 Implementation, certificates, and limits

The archive contains a new complete readable checker in `code/checker.py`. It uses tagged JSON syntax, named binders at the input boundary, and de Bruijn coordinates internally. Bound-variable renaming is therefore handled structurally. All parsing, lexical scope validation, substitution, freshness checks, computation, inference checks, and public verdict handling are in the one module; only the standard JSON parser is imported.

The input specifies a finite list of free parameter names, a goal, and a proof tree. An accepted open goal is a schematic theorem with those parameters arbitrary, not a theorem about supplied numerical values. Object goals have only Eq/D constructors. Internal Q clauses occur in contexts and as elimination majors, and are accepted there only by the hypothesis rule. The parser rejects nested object Q constructors. Unknown constructors, malformed shapes, unbound names, and Boolean aliases for hypothesis indices are rejected.

Each proof constructor implements exactly one displayed rule. CC checks both root recursors and equality of their initializer/update tails before checking the counter equality. RS performs two capture-free substitutions in the update while retaining its original body in the recursive call. DI parses its witness before allocating the fresh parameter and rejects progress at a compound matrix. DE checks the major in the old scope before extending that scope. Equality proofs cannot manufacture new parameters.

The principal suite contains 12 positive and 12 negative certificates, checked under both ordinary and optimized Python: all 48 standalone runs passed. The separate independent review passed 22 boundary checks. Positive certificates include symbolic recursion, derived successor congruence, derived initializer congruence, derived update-body congruence, and equation induction with a retained clause. Negative certificates test forbidden direct congruence, malformed endpoints, compound progress, unbound parameters, and the internal-clause boundary.

The complete readable checker measures

1617 normalized tokens, 1822 AST nodes, 6834 normalized bytes.

Its SHA-256 is

`2386eda8d133072ce9193d6256e0b6792f463dabd6afac31822e5476e6729b68`.

The producer expands the numerical congruence construction for the supplied templates. Its fixed auxiliary names are fresh for those examples; it is not a universal capture-avoiding derivation compiler. General compound transport and bi-interpretability are written proofs, not implemented general certificate transformations in this package.

10.1 What the compact counter-congruence attempts show

The new equality rules accept less arbitrary rule data, but the checker still needs general substitution for recursion and the paired binder. Consequently deleting the old equality-motive interface does not remove that shared traversal. A new rule or a narrower rule can also require another branch. The readable implementation is measured as an entire program and its size is reported rather than omitted or compared only with a conveniently large draft.

We have not produced an implementation of the new restricted calculus below the inherited frontier. The minimum-size kernels use a highly compressed representation with sorted raw arrays and intentional aliases; a comparison of their absolute source sizes with the readable implementation does not isolate the cost of CC. No claim that CC is cheaper or more expensive in an optimized matched implementation follows from that comparison. The readable implementation does not meet the requested numerical threshold.

There were also concrete attempts in the inherited compact representation:

Whole source experiment	Tokens	AST nodes	Bytes
Matched R arithmetic / general transport	296	333	704
Separate RL, CC, and QE	343	395	881
CC fused into generated local facts	341	388	892

The fusion routes a proved counter-congruence equation through the same continuation as arithmetic facts. It saves two tokens and seven AST nodes relative to the separate-CC experiment, but increases its byte count. The shared opening function is still needed for D/Q, while the old combined equality/Q branch splits into several structural checks.

These sources and 52 successful finite smoke/boundary runs are retained in `experiments/compact/`. They have not received a universal audit of the old permissive raw aliases and retain a broader unary-formula interface than the main tagged checker. They are exploratory sources, not certified replacement kernels or evidence of a new size record.

10.2 Incidental implementation records for the existing theory

The final audit found several elementary changes to the inherited generated-assumptions checker. Their full modules include the unchanged public verdict protocol and are measured by the same normalizer:

Complete program	Tokens	AST nodes	Bytes
Frozen principal baseline	281	318	685
Frozen AST baseline	283	315	710
Module import only	281	320	680
Module import / exact class	**280**	321	680
Module import / exact type	281	322	676
Module import / zero-product test	281	322	**672**
AST baseline / inverted hypothesis guard	283	**314**	751
Envelope including these variants	**280**	**314**	**672**

The import change replaces `from json import load` and `load(...)` by `import json` and `json.load(...)`. Both access the same standard parser. Every value reaching the list predicate comes from that parser or the checker's own built-in constructors. No list subclass or user-defined operator is reachable. Consequently replacing `isinstance(a, list)` by `a.__class__ is list` or `type(a) is list` preserves the branch exactly.

The shorter byte variant uses `a * 0 == []`. For an exact list it is true. For numbers, Booleans, and strings it is false, just as in the old test; this includes NaN and both floating-point infinities accepted by Python's default JSON decoder. Dictionaries and `None` raise an exception earlier, at multiplication. The old code necessarily raises on its subsequent integer comparison with those values, and no inner exception handler can recover either failure. Accepted certificates and the public ACCEPT/REJECT protocol are therefore preserved, although traceback text can differ. Physical resource exhaustion is outside this source-level equivalence claim.

The AST variant replaces the initial membership test and bare return by an inverted membership guard around the remainder of the function. Both branches have the same behavior and implicit `None` return; the global declaration stays before the guard. This removes one AST node, preserves the token count, and increases normalized bytes.

Thus the old coordinate records can be improved by one token, one AST node, or thirteen bytes (about 1.90%) in separate programs. None of these changes modifies an axiom or inference rule, and none gives a single checker strictly smaller in all three measures. They are implementation records, not the requested new mathematical axiomatization or a significant structural breakthrough. The complete variants, inherited certificates, regression checks, and source-level equivalence explanations are in `ancillary/`.

The ancillary suite passed 434 complete-program runs: seven programs, 31 inherited or malformed inputs, and ordinary/optimized Python. Of these, 310 runs exercise the five variants. All 15 guard-deletion mutation controls passed, and all seven openers agreed on 57 targeted type/scope situations. These finite checks support the source-level equivalence arguments; they do not constitute a new formal verification of the inherited arithmetic or the Python runtime.

10.3 Reproduction

From an extracted archive, run `python3reproduce.py`. The script checks the distributed hashes, executes all recorded certificate cases against the standalone checker under ordinary and optimized Python, remeasures the candidate and the frozen frontier programs, and checks the ancillary variants against their inherited fixtures. The producer is untrusted and is not imported by the checker. Run `python3build.py` to compile the report with PDFLaTeX. The standalone `ipra_counter_congruence.tex` includes the complete mathematical text and compiles without other package files. Both TeX build paths require an installed ordinary TeX distribution, but no network access or shell escape.

The symbolic checks validate concrete proof trees, including non-numeral parameters. They are not a machine verification of the metatheorems or of the Python implementation. Written soundness follows by induction on successful checking trees and the scope/substitution invariant; canonical completeness follows by serializing finite native derivations. Physical resource limits can reject very large valid certificates. No proof-assistant theorem, verified runtime, external peer review, or uniform primitive-recursive evaluator is claimed.

11 Further research questions

1. **Can counter congruence remove code rather than add a branch?** A useful next result would encode RL, CC, and numerical computation through one checked representation while deleting a trusted operation. The current result supplies the admissibility theorem, not that representation. All scope and malformed-input checks must be counted in the resulting complete program.

2. **Can the context compiler be certified once in Lean or Rocq?** The highest-value formalization is the derivation transformer proving $J_F = F$, followed by numerical and formula transport. It should expose the small dependency on numerical cases and avoid accidentally using the ambient prover's equality substitution as a primitive rule of the object calculus.

3. **What weaker instruction restores seedless completeness?** The one-coordinate bound suggests looking for a new operation whose output is unbounded in two independent inputs. An initializer is one solution; pairing, addition, or another joint data operation might offer alternatives. Each proposed basis still needs internal interpretation composites, not only standard-function expressiveness.

4. **Can a seedless basis support a relational bi-interpretation?** The negative results here exclude direct term compilers. They leave open representations by definable graphs, a changed domain, or a suitable definable quotient. Their additional proof-checking cost must be assessed rather than omitted.

5. **Can delayed witness choice be implemented economically?** A staged E introduction may repair the quantifier-order failure. A correct solution must prove the reverse formula composite for arbitrary nested formulas and make the universal/existential scope interleaving explicit.

6. **What comparison metric should govern future proposals?** Whole-program normalized source sizes are reproducible, but a coordinatewise envelope is not a single program and not an absolute lower bound. A fixed implementation grammar, semantic contract, treatment of the host runtime, and policy on aliases would make future strict-improvement claims more informative.

References

[1] Vladimir Reshetnikov and ProveIt contributors, *ProveIt*, commit `2fdcf5bf988bead821c416649f713f98b7ed8776`, accessed November 10, 2026. https://github.com/VladimirReshetnikov/ProveIt/tree/2fdcf5bf988bead821c416649f713f98b7ed8776/docs/drafts.

[2] ProveIt draft, *Atomic Transport for Intuitionistic Primitive Recursive Arithmetic*, archive `ipra_atomic_transport_2026-11-10.zip` at the pinned snapshot. Its numerical and logical bridge is adapted in this report with attribution.

[3] ProveIt draft, *Successor from Recursion*, archive `ipra_successor_recursion_2026-11-09.zip` at the pinned snapshot.

[4] The successor-recursion compiler and its use in the atomic-transport draft, cited above. The fixed-counter construction is a specialization of that compiler.

[5] ProveIt draft, *Elimination Assumptions*, archive `ipra_elimination_assumptions_2026-11-09.zip` at the pinned snapshot.

[6] ProveIt draft, *Numerical Axioms as Local Assumptions*, archive `ipra_generated_assumptions_2026-11-10.zip` at the pinned snapshot.

[7] ProveIt draft, *Fixed-Position Equality Transport*, archive `ipra_fixed_position_2026-11-09.zip`, retrieved at commit `7e694b036f89a0f16313089329555da531d6a656`. Exact archive and report hashes are in the independent novelty audit.

[8] Saeed Salehi, *Decidable Formulas of Intuitionistic Primitive Recursive Arithmetic*, Reports on Mathematical Logic 36 (2002), 55–61. Full first-order iPRA convention and atomic induction. https://rml.tcs.uj.edu.pl/rml-36/36-SALEHI.pdf.